In [ ]:
```python
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import seaborn as sns
import soundfile as sf
import marimo as mo

PROJECT_ROOT = Path("/mnt/d/Projects/PedalDSP")
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

sns.set_theme(style="whitegrid", palette="muted", font_scale=1.0)
```

KlonSim model zoo — results explorer

In [ ]:
```python
results_path_input = mo.ui.text(
    value=str(PROJECT_ROOT / "results" / "data" / "klon_sim" / "klon_sim_v1"
    label="Results pickle path",
    full_width=True,
)
results_path_input
```

Results pickle path

/mnt/d/Projects/PedalDSP/results/data/klon_sim/klon_sim_v1/results.pkl

In [ ]:
```python
"""Load results or show a helpful message if the file doesn't exist yet."""
from pedal_model.eval.results import ZooResults

_rpath = Path(results_path_input.value)
if _rpath.exists():
    zoo = ZooResults.load(_rpath)
    _status = mo.md(
        f"**Loaded** `{_rpath.name}` — {len(zoo.models)} model(s), "
        f"run at {zoo.date_run}, sr={zoo.sr} Hz"
    )
else:
    zoo = None
    _status = mo.callout(
        mo.md(
            f"Results file not found: `{_rpath}`\n\n"
            "Run the eval first:\n"
            "```\npython run_eval.py --classical-only --skip volterra\n```"
        ),
        kind="warn",
    )
_status
```

Loaded `results.pkl` — 11 model(s), run at 2026-06-07T01:18:25Z, sr=96000 Hz

Summary metrics (mean across sections)

In [ ]:

```
if zoo is None:
    mo.stop(True, mo.md("*No results loaded.*"))

df_all = zoo.to_dataframe()

_metric_cols = [c for c in df_all.columns if c not in ("type", "fit_s", "pre
_display_metrics = [
    m for m in ["ESR", "null_depth_dB", "STFT", "FR_err_dB",
                "THD_err", "THD_pattern_dist", "HP_sim", "MCD"]
    if m in _metric_cols
]

if not _display_metrics:
    mo.stop(True, mo.md("*No metric columns found in results.*"))

_df_heat = df_all[_display_metrics].copy()

# Normalise each column to [0, 1] for the heatmap colour scale
_df_norm = (_df_heat - _df_heat.min()) / (_df_heat.max() - _df_heat.min() +

fig_heat, ax_heat = plt.subplots(figsize=(max(8, len(_display_metrics) * 1.1
sns.heatmap(
    _df_norm,
    ax=ax_heat,
    annot=_df_heat.round(3),
    fmt=".3g",
    cmap="RdYlGn_r",
    linewidths=0.5,
    cbar_kws={"label": "normalised (0=best, 1=worst)"},
)
ax_heat.set_title("Model zoo — mean metrics across all sections")
ax_heat.set_xlabel("")
ax_heat.set_ylabel("")
plt.tight_layout()
mo.mpl.interactive(fig_heat)
```



In [ ]:

```python
if zoo is None:
    mo.stop(True)

_model_names = list(zoo.models.keys())
model_selector = mo.ui.multiselect(
    _model_names,
    value=_model_names[:min(4, len(_model_names))],
    label="Models to compare",
)

# Collect section labels from first model that has sections
_section_names: list[str] = []
for _mr in zoo.models.values():
    if _mr.metrics_per_section:
        _section_names = list(_mr.metrics_per_section.keys())
        break

section_selector = mo.ui.dropdown(
    _section_names,
    value=_section_names[0] if _section_names else None,
    label="Section for waveform plot",
)

mo.hstack([model_selector, section_selector])
```

Models to compare 4 selected

Section for waveform plot val_sweep_l-24

Waveform comparison

In [ ]:

```
if zoo is None or not model_selector.value or section_selector.value is None
    mo.stop(True, mo.md("*Select at least one model and a section.*"))

from pedal_model.signals.manifest import Manifest

# Find the section bounds from the first available model's section list
_manifest_path = (
    zoo.config.get("params_path", "").replace("params.json", "")
    or ""
)

# Fall back: derive manifest path from klon_sim_dir
_params = zoo.config.get("cfg", "")
# Try to find manifest path through the ZooResults
_sections_ref = None
for _mr in zoo.models.values():
    if _mr.metrics_per_section and section_selector.value in _mr.metrics_per
        break

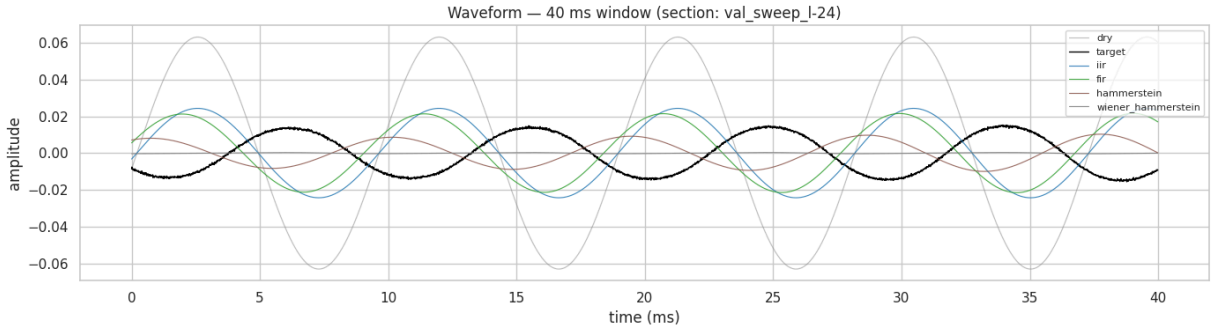
# We don't have the manifest object in results — use sample positions from m
# Instead, show a fixed 50ms window from the approximate start.
sr = zoo.sr
_window_s = 0.04  # 40 ms
_start_s = 2.0    # skip first 2 s (silence gap)
_start = int(_start_s * sr)
_end = _start + int(_window_s * sr)

dry = zoo.dry_val[_start:_end]
wet = zoo.wet_val[_start:_end]
t_ms = 1000.0 * (np.arange(len(dry)) / sr)

fig_wf, ax_wf = plt.subplots(figsize=(14, 4))
ax_wf.plot(t_ms, dry, color="gray", alpha=0.5, lw=0.8, label="dry")
ax_wf.plot(t_ms, wet, color="black", lw=1.0, label="target")

_cmap = plt.cm.tab10
for _i, _key in enumerate(model_selector.value):
    if _key in zoo.models:
        _pred = zoo.models[_key].prediction[_start:_end]
        ax_wf.plot(t_ms, _pred, lw=0.8, alpha=0.85,
                   color=_cmap(_i / max(1, len(model_selector.value))),
                   label=_key)

ax_wf.set_xlabel("time (ms)")
ax_wf.set_ylabel("amplitude")
ax_wf.set_title(f"Waveform — 40 ms window (section: {section_selector.value}
ax_wf.legend(loc="upper right", fontsize=8)
plt.tight_layout()
mo.mpl.interactive(fig_wf)
```



Null-test depth (dB) — higher is better

In [ ]:

```
if zoo is None or not model_selector.value:
    mo.stop(True, mo.md("*No models selected.*"))

_selected = [k for k in model_selector.value if k in zoo.models]
if not _selected:
    mo.stop(True)

# Collect per-section null depths
_section_labels: list[str] = []
_model_depths: dict[str, list[float]] = {k: [] for k in _selected}

for _sec_label in (list(zoo.models[_selected[0]].metrics_per_section.keys())
                   if _selected else []):
    _section_labels.append(_sec_label)
    for _key in _selected:
        val_ = zoo.models[_key].metrics_per_section.get(_sec_label, {}).get(
        _model_depths[_key].append(val_ if not (val_ != val_) else 0.0)  # r

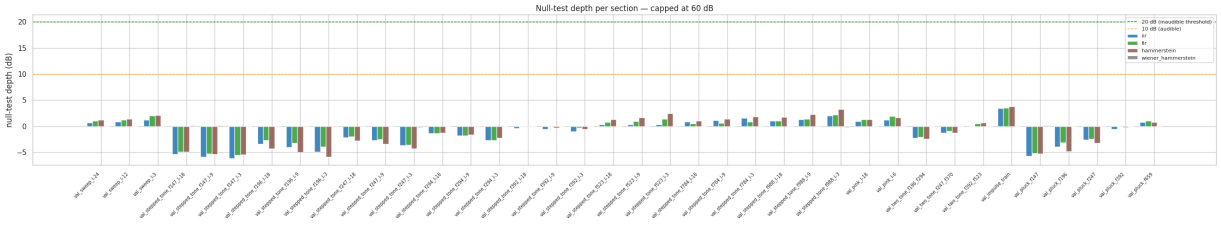
_n_sec = len(_section_labels)
if _n_sec == 0:
    mo.stop(True, mo.md("*No sections found.*"))

_x = np.arange(_n_sec)
_width = 0.8 / max(len(_selected), 1)
_cmap = plt.cm.tab10

fig_null, ax_null = plt.subplots(figsize=(max(10, _n_sec * 0.7), 5))
for _i, _key in enumerate(_selected):
    _vals = _model_depths[_key]
    _finite = [v if v < 1e6 else 60.0 for v in _vals]  # cap inf at 60 dB
    ax_null.bar(
        _x + _i * _width - 0.4,
        _finite,
        _width,
        label=_key,
        color=_cmap(_i / max(1, len(_selected))),
        alpha=0.85,
    )

ax_null.axhline(20, color="green", ls="--", lw=1, label="20 dB (inaudible th
ax_null.axhline(10, color="orange", ls="--", lw=1, label="10 dB (audible)")
```

```
ax_null.set_xticks(_x)
ax_null.set_xticklabels(_section_labels, rotation=45, ha="right", fontsize=7
ax_null.set_ylabel("null-test depth (dB)")
ax_null.set_title("Null-test depth per section — capped at 60 dB")
ax_null.legend(fontsize=8)
plt.tight_layout()
mo.mpl.interactive(fig_null)
```



ESR per section

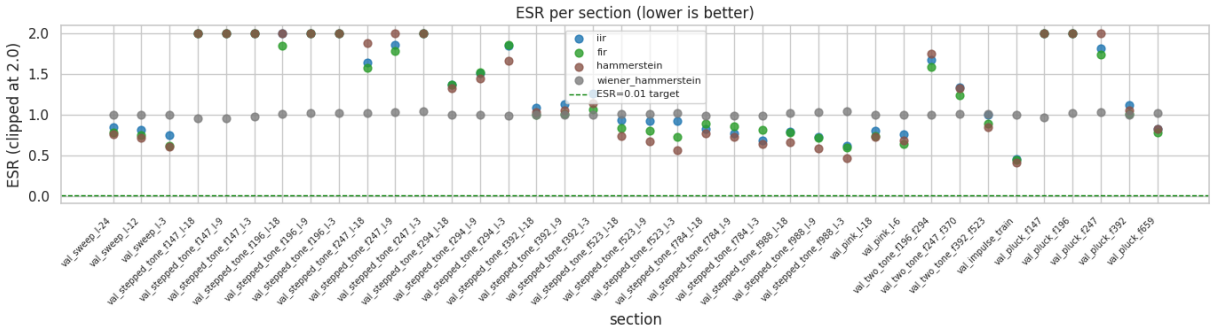
In [ ]:
```
if zoo is None or not model_selector.value:
    mo.stop(True)

_selected = [k for k in model_selector.value if k in zoo.models]
_rows = []
for _key in _selected:
    for _sec, _metrics in zoo.models[_key].metrics_per_section.items():
        _rows.append({"model": _key, "section": _sec, "ESR": _metrics.get("E

_df_esr = pd.DataFrame(_rows)
if _df_esr.empty:
    mo.stop(True, mo.md("*No ESR data.*"))

_cmap = plt.cm.tab10
fig_esr, ax_esr = plt.subplots(figsize=(14, 4))
for _i, _key in enumerate(_selected):
    _sub = _df_esr[_df_esr["model"] == _key]
    ax_esr.scatter(
        _sub["section"],
        _sub["ESR"].clip(upper=2.0),
        label=_key,
        s=40,
        alpha=0.8,
        color=_cmap(_i / max(1, len(_selected))),
    )

ax_esr.axhline(0.01, color="green", ls="--", lw=1, label="ESR=0.01 target")
ax_esr.set_xlabel("section")
ax_esr.set_ylabel("ESR (clipped at 2.0)")
ax_esr.set_title("ESR per section (lower is better)")
ax_esr.legend(fontsize=8)
plt.xticks(rotation=45, ha="right", fontsize=7)
plt.tight_layout()
mo.mpl.interactive(fig_esr)
```



KlonDDSP: circuit vs. ML correction breakdown

In [ ]:
```
if zoo is None:
    mo.stop(True)

_klon_result = zoo.models.get("klon_ddsp")
if _klon_result is None:
    mo.md("*klon_ddsp not in results — run eval with neural models.*")
elif _klon_result.ddsp_analysis is None:
    mo.md(
        "*DDSP fraction analysis not stored in results.*\n\n"
        "Re-run `run_eval.py` to populate this field."
    )
else:
    _a = _klon_result.ddsp_analysis
    mo.md(
        f"""
**KlonDDSP decomposition:**
| Metric | Value |
|--------|-------|
| DDSP stage ESR | {_a['ddsp_esr']:.4f} |
| Full model ESR | {_a['full_esr']:.4f} |
| Correction RMS ratio | {_a['correction_rms_ratio']:.3f} |
| **DDSP fraction** | **{_a['ddsp_fraction']:.1%}** |

*DDSP fraction* = how much of the total variance explained by the full model
is already captured by the physics-informed DDSP circuit alone.
A high value (→100 %) means the Klon circuit topology accounts for most of
the pedal's behaviour; a low value means the GRU correction is doing the
heavy lifting (suggesting the topology assumptions are wrong or incomplete).
"""
    )
```

Timing

In [ ]:
```
if zoo is None:
    mo.stop(True)
_timing = df_all[["fit_s", "predict_s"]].copy()
_timing.columns = ["fit / train (s)", "predict (s)"]
mo.ui.table(_timing.reset_index().rename(columns={"model": "Model"}))
```

	_marimo_row_id	Model	fit / train (s)	predict (s)
0	0	iir	0.021354	0.363453
1	1	fir	0.019148	0.706283
2	2	hammerstein	0.137579	0.905255
3	3	wiener_hammerstein	0.115117	1.300166
4	4	volterra	3.541049	17.113088
...	...	...	...	...
6	6	lstm	1.777740	5.098000
7	7	gru	1.196228	108.980565
8	8	tcn	4.369325	16.428447
9	9	ddsp	1.605241	1.809729
10	10	klon_ddsp	1.261640	106.882635

11 rows × 4 columns